

Resumo didático completo de **Notas.txt**

Fonte principal: [Project_MySQL/src/Notas.txt](#)

1. Visão geral

O arquivo apresenta uma introdução prática à administração e à otimização avançada do MySQL. A proposta é abandonar a visão do banco como uma simples “planilha” e entendê-lo como um sistema transacional composto por mecanismos de armazenamento, índices, logs, controle de concorrência, recuperação e replicação.

Os eixos centrais são o mecanismo **InnoDB**, os índices baseados em árvores B/B+, o planejamento de execução de consultas, o **MVCC** — controle de concorrência por múltiplas versões —, o locking, os backups, o *Point-in-Time Recovery* (PITR), o monitoramento e o Percona Toolkit.

A ideia mais importante do material é: desempenho não depende apenas da quantidade de dados, mas principalmente de como o MySQL localiza, lê, bloqueia e grava as linhas.

2. SELECT avançado e análise de execução

2.1 O problema não é apenas o volume de dados

Uma tabela com milhões de registros pode responder rapidamente quando o MySQL consegue usar um índice seletivo. Por outro lado, uma consulta aparentemente simples pode ficar lenta se obrigar o servidor a examinar muitas linhas.

O exemplo usa uma tabela `pedidos` com um índice composto sobre `(client_id, data_criacao)`. Esse índice é adequado para consultas que filtram primeiro o cliente e depois fazem uma busca por intervalo de data.

SQL

```
CREATE TABLE pedidos (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  client_id INT NOT NULL,  
  data_criacao DATETIME NOT NULL,  
  valor DECIMAL(10,2),  
  status TINYINT,  
  INDEX idx_client_data (client_id, data_criacao)  
);
```

2.2 Por que aplicar uma função na coluna pode quebrar o uso do índice

A consulta abaixo aplica `YEAR()` diretamente sobre a coluna indexada:

SQL

```
SELECT *
FROM pedidos
WHERE client_id = 10
AND YEAR(data_criacao) = 2025;
```

Isso dificulta uma busca direta pela árvore do índice, porque o MySQL precisa calcular o ano de cada valor antes de compará-lo. Consequentemente, ele pode examinar todas — ou grande parte — das linhas do cliente.

A forma preferível é transformar o valor pesquisado em um intervalo, mantendo a coluna “livre” de funções:

SQL

```
EXPLAIN ANALYZE
SELECT client_id, data_criacao
FROM pedidos
WHERE client_id = 10
AND data_criacao >= '2025-01-01'
AND data_criacao < '2026-01-01';
```

O uso de intervalo permite um **range scan**, isto é, uma leitura apenas da região relevante do índice.

2.3 EXPLAIN e EXPLAIN ANALYZE

`EXPLAIN` mostra o plano estimado pelo otimizador. `EXPLAIN ANALYZE`, disponível no MySQL 8.0, executa a consulta e apresenta tempos e quantidades observadas na prática. Ele ajuda a comparar o plano esperado com o comportamento real.

Um item importante é `Using index`. Quando isso representa um **covering index** — índice cobertor —, todas as colunas necessárias à consulta estão no próprio índice. O servidor consegue responder sem consultar as páginas de dados da tabela.

Por isso, neste caso:

SQL

```
SELECT client_id, data_criacao
```

pode ser muito mais eficiente do que:

```
SQL
```

```
SELECT *
```

O `SELECT *` solicita colunas que não estão no índice, obrigando o InnoDB a localizar a linha completa na estrutura de dados. Essas buscas adicionais são frequentemente chamadas de *bookmark lookups* ou buscas de retorno à tabela.

3. INSERT em massa e transações

3.1 Agrupar operações reduz custo

Executar um `COMMIT` para cada linha é caro, pois cada confirmação pode exigir sincronização do log em disco. Em cargas grandes, a recomendação do arquivo é agrupar os registros em lotes, por exemplo, mil linhas por transação:

```
SQL
```

```
START TRANSACTION;
```

```
INSERT INTO pedidos (client_id, data_criacao, valor, status)
```

```
VALUES
```

```
  (1, NOW(), 100.50, 1),
```

```
  (2, NOW(), 200.75, 1);
```

```
-- demais linhas do lote
```

```
COMMIT;
```

O tamanho ideal do lote depende da memória, do hardware, da quantidade de índices, das chaves estrangeiras e dos requisitos de recuperação. Lotes excessivamente grandes também podem criar transações difíceis de reverter.

3.2 Parâmetros de desempenho: usar com cautela

O texto menciona a desativação temporária de verificações para cargas controladas. Os nomes corretos das variáveis são, em geral, `unique_checks` e `foreign_key_checks` :

```
SQL
```

```
SET autocommit = 0;  
SET unique_checks = 0;  
SET foreign_key_checks = 0;
```

Essas opções só devem ser usadas quando os dados foram previamente validados e o contexto é controlado. Desativar chaves estrangeiras pode permitir inconsistências; desativar verificações de unicidade pode permitir problemas que só aparecerão posteriormente.

O parâmetro citado no arquivo é `innodb_flush_log_at_trx_commit`. O valor `1` prioriza durabilidade máxima. O valor `2` escreve o log no sistema operacional a cada *commit*, mas sincroniza o arquivo em intervalos periódicos, aumentando o desempenho e reduzindo a garantia de durabilidade diante de uma falha abrupta do sistema operacional ou da máquina.

3.3 UPSERT com ON DUPLICATE KEY UPDATE

Quando o objetivo é inserir ou atualizar um registro conforme a existência de uma chave, pode-se evitar o padrão “consultar primeiro, inserir depois” :

SQL

```
INSERT INTO pedidos (id, valor, status)  
VALUES (1001, 150.00, 5)  
ON DUPLICATE KEY UPDATE  
  valor = VALUES(valor),  
  status = VALUES(status);
```

Esse padrão é útil em processos de carga e em *data warehouses*. Em versões recentes do MySQL, convém verificar a sintaxe recomendada para substituir `VALUES(coluna)` por aliases de linha, conforme a versão utilizada.

4. DELETE, TRUNCATE e remoção em lotes

4.1 Perigos de um DELETE gigante

Uma exclusão muito grande em uma única transação pode gerar um *undo log* volumoso, aumentar o log binário, manter locks por muito tempo, elevar o uso de I/O e dificultar um eventual rollback.

A estratégia recomendada é excluir em lotes pequenos:

SQL

```
DELETE FROM pedidos
WHERE data_criacao < '2020-01-01'
LIMIT 1000;
```

A operação deve ser repetida até que não restem linhas, preferencialmente em períodos de baixa utilização. O tamanho do lote e a pausa entre lotes devem ser ajustados mediante observação de locks, replicação, I/O e latência.

4.2 pt-archiver

O `pt-archiver`, do Percona Toolkit, automatiza a remoção controlada. Ele pode arquivar registros em outra tabela, excluir a origem em lotes e inserir pausas para limitar o impacto sobre o banco.

4.3 Diferença entre DELETE e TRUNCATE

`DELETE` é uma operação DML e pode ser filtrado por condição. `TRUNCATE TABLE` é tratado como DDL, remove o conteúdo da tabela de forma muito mais rápida e normalmente provoca *commit* implícito. Em razão dessas características, não deve ser considerado equivalente a um `DELETE` transacional comum e não deve ser usado quando a operação precisar de rollback.

5. Backup, binlog e recuperação até um ponto no tempo

5.1 Backup lógico com mysqldump

Para tabelas InnoDB, o arquivo recomenda um dump consistente usando `--single-transaction`. Essa opção obtém uma visão consistente sem bloquear, em condições normais, as operações de leitura e escrita do InnoDB durante todo o dump.

A opção `--master-data=2` registra no arquivo de backup as coordenadas do binlog no momento associado ao snapshot. O comando apresentado é:

Bash

```
mysqldump -u root -p \
--single-transaction \
--master-data=2 \
--routines \
--triggers \
--events \
--all-databases > backup_completo.sql
```

É importante lembrar que `--single-transaction` não torna automaticamente consistentes objetos não transacionais, nem substitui a necessidade de planejar procedimentos para tabelas, eventos, usuários, arquivos externos e configurações.

5.2 O que significam `MASTER_LOG_FILE` e `MASTER_LOG_POS`

No arquivo `.sql`, pode aparecer uma linha semelhante a:

SQL

```
-- CHANGE MASTER TO  
-- MASTER_LOG_FILE='mysql-bin.000042', MASTER_LOG_POS=1879;
```

Os valores são exemplos e variam em cada servidor. `MASTER_LOG_FILE` identifica o arquivo de binlog. `MASTER_LOG_POS` identifica a posição dentro desse arquivo. Juntos, eles indicam o ponto a partir do qual uma réplica deve continuar lendo as alterações, porque as operações anteriores já estão incluídas no backup.

A opção `--master-data=2` deixa a instrução comentada para impedir uma execução acidental. Com `--master-data=1`, a linha pode ser gravada de forma ativa, dependendo da versão e do fluxo usado.

Para descobrir os valores reais, procure a linha no dump:

Bash

```
grep -i "CHANGE MASTER" backup_completo.sql
```

Também é possível usar:

Bash

```
head -n 100 backup_completo.sql | grep -i "log_file"
```

Se a linha não existir, o dump provavelmente foi produzido sem a opção necessária. Nesse caso, a replicação tradicional por arquivo e posição não pode ser iniciada com segurança a partir daquele backup, a menos que exista outra fonte confiável de coordenadas. Se GTID estiver habilitado, pode haver uma alternativa baseada em identificadores globais.

5.3 PITR — Point-in-Time Recovery

O PITR combina um backup completo com os eventos posteriores armazenados no binlog. Se o backup foi feito às 22h00 e o problema ocorreu às 22h15, o procedimento geral é:

Bash

```
mysql -u root -p < backup_completo.sql

mysqlbinlog \
  --start-position=456 \
  --stop-datetime="2026-08-15 22:14:59" \
  binlog.000123 | mysql -u root -p
```

A posição inicial deve ser extraída do próprio backup, e a data final deve ser escolhida conforme o instante imediatamente anterior ao incidente. Em um ambiente real, a operação precisa ser validada, isolada e testada antes da execução definitiva.

5.4 Backup físico com XtraBackup

Para bancos muito grandes, o arquivo recomenda avaliar o Percona XtraBackup. Diferentemente do dump lógico, ele trabalha diretamente com os arquivos físicos do InnoDB e pode reduzir significativamente o tempo de restauração. A escolha depende do tamanho do banco, da janela de recuperação, da infraestrutura e do procedimento de validação.

6. Monitoramento e teste de desempenho

6.1 Slow Query Log

O *Slow Query Log* registra consultas que ultrapassam determinado tempo ou que não usam índices:

SQL

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 0.5;
SET GLOBAL log_queries_not_using_indexes = 'ON';
```

O log deve ser usado para coletar evidências, não apenas para confirmar uma impressão subjetiva de lentidão. É preciso observar tempo, linhas examinadas, linhas retornadas, frequência e concorrência.

6.2 Schema sys

O schema `sys`, disponível nas versões modernas do MySQL, oferece visões úteis para diagnosticar consumo e índices. O arquivo cita consultas para encontrar as instruções com maior latência total e os índices aparentemente não utilizados:

SQL

```
SELECT *  
FROM sys.schema_unused_indexes;
```

Os nomes exatos das visões devem ser conferidos na versão instalada; o material contém alguns nomes digitados de forma imprecisa.

6.3 Teste de carga com `mysqlslap`

O `mysqlslap` simula concorrência e repetições. O objetivo é verificar se o tempo médio cresce com o número de conexões:

Bash

```
mysqlslap \  
  --user=root \  
  --password=senha \  
  --host=localhost \  
  --concurrency=50 \  
  --iterations=10 \  
  --create-schema=meu_banco \  
  --query="SELECT client_id, COUNT(*)  
          FROM pedidos  
          WHERE data_criacao > '2025-01-01'  
          GROUP BY client_id;"
```

Crescimento acentuado pode indicar contenção de locks, falta de índices, saturação de CPU, I/O insuficiente ou outros gargalos. O teste deve ser realizado em ambiente controlado, pois pode sobrecarregar o banco.

6.4 Buffer Pool

O *buffer pool* é a principal área de memória do InnoDB. Ele mantém páginas de dados e índices em RAM, reduzindo leituras em disco. Uma consulta pode ser otimizada no plano, mas ainda sofrer com falta de memória ou baixa taxa de acerto do cache.

O arquivo sugere consultar estatísticas por tabela:

SQL

```
SELECT *  
FROM sys.innodb_buffer_stats_by_table  
WHERE object_schema = 'meu_banco'  
      AND object_name = 'pedidos';
```


A configuração de `innodb_buffer_pool_size` deve considerar a RAM disponível, outros serviços no mesmo servidor, picos de utilização e necessidade de evitar swap. A regra de 70% a 80% apresentada é uma heurística, não uma lei universal.

7. Laboratório de degradação de performance

O laboratório simula uma tabela `orders` com milhões de registros e três problemas combinados:

Problema	Consequência
<code>YEAR(created_at)</code> no filtro	Dificulta o uso eficiente do índice de data.
<code>OFFSET 500000</code>	Obriga o MySQL a localizar e descartar centenas de milhares de linhas.
<code>FOR UPDATE</code> sem índice seletivo	Pode ampliar a quantidade de registros e intervalos bloqueados.

A consulta problemática é semelhante a:

SQL

```
SELECT *
FROM orders
WHERE YEAR(created_at) = 2026
      AND status IN (1, 2)
ORDER BY id
LIMIT 10 OFFSET 500000;
```

O diagnóstico deve combinar o slow log, `EXPLAIN ANALYZE`, `SHOW ENGINE INNODB STATUS` e contadores de leitura da sessão. A métrica importante não é apenas o tempo final, mas também quantas linhas foram examinadas para retornar somente dez.

7.1 Correção do filtro de data

Substitua a função na coluna por limites de data:

SQL

```
SELECT *
FROM orders
WHERE created_at >= '2026-01-01 00:00:00'
      AND created_at < '2027-01-01 00:00:00'
```

```
AND status IN (1, 2)
ORDER BY id
LIMIT 10 OFFSET 500000;
```

O intervalo com limite superior exclusivo é especialmente útil para evitar problemas com frações de segundo. O índice ideal depende da consulta completa e da distribuição dos dados; não se deve presumir que o índice existente será sempre escolhido.

7.2 Correção da paginação com Seek Method

`OFFSET` é cada vez mais caro à medida que a página avança. A alternativa é enviar ao servidor o último identificador visto e buscar a página seguinte a partir dele:

SQL

```
SELECT *
FROM orders
WHERE created_at >= '2026-01-01'
      AND created_at < '2027-01-01'
      AND status IN (1, 2)
      AND id > 984532
ORDER BY id
LIMIT 10;
```

Esse método, chamado **seek pagination** ou **cursor pagination**, evita descartar centenas de milhares de registros. Para ordenações mais complexas, o cursor precisa carregar todas as colunas necessárias à ordenação, e o índice deve refletir essa ordenação.

7.3 Correção do locking

Para o processamento concorrente:

SQL

```
START TRANSACTION;

SELECT id, total
FROM orders
WHERE status = 1
FOR UPDATE;
```

O arquivo recomenda um índice como:

SQL

```
CREATE INDEX idx_status_id ON orders(status, id);
```

Com um índice adequado, o InnoDB consegue localizar os registros relevantes com mais precisão. Ainda assim, o comportamento exato de locks depende do nível de isolamento, do plano escolhido e do tipo de busca. Em especial, leituras bloqueantes podem usar locks de registro e *next-key locks* em determinadas condições.

7.4 Tipo de dado e conversão implícita

Se `customer_id` armazena números como `VARCHAR`, a comparação deve usar uma string:

SQL

```
EXPLAIN SELECT * FROM orders WHERE customer_id = '12345';
```

Comparar com um número:

SQL

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 12345;
```

pode provocar conversão implícita e impedir o uso eficiente do índice. A regra geral é comparar valores com o mesmo tipo e formato armazenados na coluna.

8. Percona Toolkit

O Percona Toolkit é uma coleção de ferramentas de linha de comando para diagnóstico, manutenção, alterações de esquema, auditoria de replicação, arquivamento e análise de índices. O material compara o conjunto a um “canivete suíço” do administrador de bancos de dados.

8.1 Ferramentas principais

Ferramenta	Finalidade principal
<code>pt-query-digest</code>	Agrupar e analisar queries lentas, destacando as que mais consomem tempo e linhas.
<code>pt-online-schema-change</code>	Alterar tabelas grandes com impacto reduzido e sem o bloqueio prolongado de um <code>ALTER</code> tradicional.

pt-table-checksum	Verificar diferenças entre dados do mestre e das réplicas.
pt-table-sync	Corrigir divergências identificadas na replicação.
pt-archiver	Arquivar ou remover dados antigos em lotes controlados.
pt-index-usage	Identificar índices que não aparecem no uso observado pelas queries.
pt-mysql-summary	Gerar uma visão geral da configuração, status, hardware e ambiente do MySQL.

O `pt-query-digest` transforma um log extenso em um relatório priorizado. O `pt-online-schema-change` cria uma tabela auxiliar, copia os dados em lotes, acompanha alterações por triggers e realiza uma troca final. O `pt-table-checksum` calcula verificações comparáveis entre servidores, enquanto o `pt-table-sync` pode corrigir as diferenças.

Antes de executar ferramentas que alteram dados, o arquivo recomenda fazer backup, testar em desenvolvimento, usar `--print` ou `--dry-run` quando disponível e escolher um período de baixa demanda.

9. Replicação no MySQL

9.1 Conceito

Na replicação, a **fonte** registra alterações no binlog. A **réplica** lê esses eventos, armazena-os no relay log e os aplica localmente. O objetivo é manter cópias sincronizadas e possibilitar distribuição de leituras, backups em uma réplica, análises pesadas fora da fonte e recuperação diante de falhas.

9.2 Configuração da fonte

O arquivo sugere, na seção `[mysqld]`, parâmetros semelhantes a:

Plain Text

```
server-id = 1
log-bin = /var/log/mysql/mysql-bin
binlog-format = ROW
binlog_row_image = MINIMAL
```

```
sync_binlog = 1
innodb_flush_log_at_trx_commit = 1
```

`server-id` deve ser único. `log-bin` habilita o binlog. `ROW` registra alterações por linha, o que tende a ser mais previsível para replicação. `sync_binlog = 1` e `innodb_flush_log_at_trx_commit = 1` priorizam durabilidade, embora possam aumentar o custo de escrita.

Depois de reiniciar o serviço, cria-se um usuário dedicado:

SQL

```
CREATE USER 'repl'@'IP_DA_REPLICA' IDENTIFIED BY 'senha-forte';
GRANT REPLICATION REPLICATION ON *.* TO 'repl'@'IP_DA_REPLICA';
FLUSH PRIVILEGES;
```

O privilégio e a sintaxe exatos devem ser confirmados na versão utilizada. Restringir o host é mais seguro do que permitir `%`.

As coordenadas atuais são obtidas com:

SQL

```
SHOW MASTER STATUS;
```

9.3 Configuração da réplica

A réplica deve ter outro identificador e pode usar:

Plain Text

```
server-id = 2
relay-log = /var/log/mysql/relay-bin
read_only = ON
log_bin = /var/log/mysql/mysql-bin
log_replica_updates = ON
```

O relay log armazena eventos recebidos. `read_only` reduz o risco de escritas acidentais, embora contas administrativas possam ter privilégios que contornem essa proteção.

`log_replica_updates` permite registrar eventos recebidos, recurso necessário em topologias em cascata.

Em MySQL 8.0.23 ou superior, a nomenclatura recomendada é:

SQL

```

CHANGE REPLICATION SOURCE TO
  SOURCE_HOST = 'IP_DA_FONTE',
  SOURCE_PORT = 3306,
  SOURCE_USER = 'repl',
  SOURCE_PASSWORD = 'senha-forte',
  SOURCE_LOG_FILE = 'mysql-bin.000001',
  SOURCE_LOG_POS = 154;

START REPLICA;
SHOW REPLICA STATUS\G

```

Em versões antigas, aparecem os comandos `CHANGE MASTER TO` , `START SLAVE` e `SHOW SLAVE STATUS` . O material mistura as nomenclaturas antiga e nova; a escolha deve ser compatível com a versão instalada.

9.4 Como verificar a replicação

Os principais campos são:

Campo	Interpretação esperada
<code>Replica_IO_Running</code> ou nomenclatura equivalente	O thread de I/O está conectado e recebendo eventos.
<code>Replica_SQL_Running</code> ou nomenclatura equivalente	O thread SQL está aplicando eventos.
<code>Seconds_Behind_Source</code> ou campo equivalente	Atraso aproximado em relação à fonte; zero normalmente indica sincronização.
<code>Last_IO_Error</code>	Problemas de conexão, autenticação ou leitura do binlog.
<code>Last_SQL_Error</code>	Falha ao aplicar algum evento no banco.
<code>Read_Source_Log_Pos</code> e <code>Exec_Source_Log_Pos</code>	Diferença entre o que foi lido e o que já foi aplicado.

Um valor `NULL` no atraso pode significar que a réplica não está executando ou perdeu a conexão. O status deve ser interpretado em conjunto com os erros, pois “zero” isoladamente não prova que todos os dados estejam corretos.

10. Replicação baseada em GTID

O GTID atribui um identificador global a cada transação. Com isso, a réplica pode localizar automaticamente o ponto adequado de continuidade, reduzindo a dependência de nomes de arquivos e posições manuais.

A ativação envolve, entre outros parâmetros:

Plain Text

```
gtid_mode = ON
enforce_gtid_consistency = ON
```

Na réplica, utiliza-se uma configuração baseada em auto-posicionamento, conforme a sintaxe da versão:

SQL

```
CHANGE REPLICATION SOURCE TO
SOURCE_HOST = 'IP_DA_FONTE',
SOURCE_USER = 'repl',
SOURCE_PASSWORD = 'senha-forte',
SOURCE_AUTO_POSITION = 1;
```

O planejamento de GTID deve ser feito antes da produção, considerando servidores já existentes, transações incompatíveis, failover e procedimentos de recuperação.

11. Modalidades e recursos avançados de replicação

Modalidade ou recurso	Ideia central	Uso típico
Assíncrona	A fonte não espera confirmação da réplica.	Maior desempenho.
Semissíncrona	A fonte aguarda ao menos o recebimento do evento por uma réplica.	Equilíbrio entre desempenho e segurança.
Síncrona	A confirmação depende das réplicas configuradas.	Maior proteção contra perda, com maior custo.
Delayed replication	A réplica é mantida deliberadamente atrasada.	Recuperação contra erros humanos.
Filtros de replicação	Apenas determinados bancos ou tabelas são replicados.	Reduzir escopo e carga, com planejamento cuidadoso.

O atraso deliberado, por exemplo, pode ser configurado para uma hora com `SOURCE_DELAY = 3600`, conforme a sintaxe suportada. Ele cria uma janela para recuperar dados antes de um erro humano ser reproduzido na réplica.

12. Checklist operacional consolidado

Antes de diagnosticar uma consulta, examine o plano com `EXPLAIN` ou `EXPLAIN ANALYZE`, verifique a seletividade dos índices e observe linhas examinadas versus linhas retornadas. Evite funções sobre colunas indexadas, mantenha os tipos de comparação coerentes e prefira paginação por cursor a offsets muito grandes.

Em cargas e exclusões, agrupe operações em lotes, monitore locks e replicação, e não desative verificações sem validar previamente os dados. Para backups, mantenha cópias testadas, binlogs preservados e um procedimento de restauração reproduzível. Um backup nunca deve ser considerado confiável apenas porque foi criado; é preciso testar a restauração.

Na replicação, use `server-id` único, usuário restrito, rede protegida, binlog configurado, monitoramento contínuo e validação do status. Ao alterar dados ou estruturas com ferramentas externas, faça backup, ensaie em desenvolvimento, leia o modo de simulação e execute em janela controlada.

13. Correções e ressalvas técnicas do arquivo original

O material é didático, mas contém erros de digitação, nomes de comandos e exemplos que precisam ser revisados antes de qualquer execução. Entre os problemas mais evidentes estão `DATE_TIME` em vez de `DATETIME`, `EXPLAIN ANALIZE` em vez de `EXPLAIN ANALYZE`, `innodb_at_trx_comit` em vez de `innodb_flush_log_at_trx_commit`, `DATE_STUB` em vez de uma função válida de data, `relay_only` em vez de `read_only`, além de divergências entre `CHANGE MASTER TO` e `CHANGE REPLICATION SOURCE TO`.

Também há afirmações simplificadas que dependem do plano, da versão, do nível de isolamento e da distribuição dos dados. Por exemplo, um índice em `status` não é automaticamente inútil só porque possui poucos valores; sua utilidade depende da seletividade combinada, da consulta e da cardinalidade efetiva. Da mesma forma, `EXPLAIN` não “mente”: ele pode apresentar um plano estimado ou uma leitura de índice que não significa necessariamente um *seek* altamente seletivo.

Portanto, o arquivo deve ser utilizado como roteiro conceitual e laboratório de estudo, não como um conjunto de comandos para copiar diretamente em produção.

14. Síntese final

O conteúdo conduz o leitor do nível de usuário SQL ao raciocínio de arquitetura e operação de bancos. A sequência lógica é: entender como os dados são armazenados; observar como o otimizador acessa as linhas; reduzir leituras desnecessárias; controlar o tamanho das transações; remover dados em lotes; manter backups restauráveis; usar binlogs para PITR; medir o comportamento com logs e testes de carga; e garantir continuidade por meio de replicação.

As duas regras práticas que resumem o laboratório são:

Não aplique transformações à coluna indexada no `WHERE` ; transforme o valor comparado e preserve a possibilidade de uso do índice.

Para paginação profunda, evite `OFFSET` e use um cursor baseado na última chave vista, com índice compatível com a ordenação.

Referência

[1] Project_MySQL — Notas.txt